

Tarea Final de la Unidad PROG07

Realizar un ejercicio en JAVA sobre la utilización avanzada de clases.

Entrega

Deberás crear una carpeta comprimida en formato ZIP con el nombre:

Apellido_Nombre_Tarea_Final_PROG07

Asegúrate que el nombre no contenga la letra ñ, tildes ni caracteres especiales extraños. Así por ejemplo la alumna **Begoña Sánchez Mañas para unidad 07de Programación**, debería nombrar esta tarea como:

sanchez_manas_begona_Tarea_Final_PROG07.zip

Dentro de esta carpeta incluye todas las clases necesarias para la ejecución del programa. Finalmente, sube el archivo comprimido en la tarea correspondiente del aula virtual.

Enunciado del Problema

Título: Gestión de Empleados

Descripción:

Se debe desarrollar un sistema en Java que permita gestionar distintos tipos de empleados de una empresa, aplicando los principios de **herencia**, **clases abstractas**, **interfaces** y el uso de **ArrayList** para almacenar la información.

El programa permitirá:

- Registrar empleados de distintas categorías.
- Calcular el salario total según el tipo de empleado.
- Mostrar información detallada de cada empleado.

Especificaciones

Clase Abstracta Empleado

Atributos:

- nombre (String)
- edad (int)
- salarioBase (double)

Constructores:

- Constructor parametrizado para inicializar los atributos.

Métodos abstractos:

- `calcularSalario(): double`
(Debe implementarse de forma específica en cada tipo de empleado.)

Otros métodos:

- `mostrarInformacion(): String`
(Devuelve una cadena con la información general del empleado incluyendo el salario calculado.)

Clases hijas de Empleado

Cada tipo de empleado extiende Empleado, implementa Imprimible y sobrescribe `calcularSalario()`.

Socio

- Atributo adicional: `stockOptions (int)`
- $\text{Salario} = \text{salarioBase} + (\text{stockOptions} * 122)$

GeneralManager

- Atributo adicional: `bono (double)`
- $\text{Salario} = \text{salarioBase} + \text{bono}$

Manager

- Atributo adicional: `bono (double)`
- $\text{Salario} = \text{salarioBase} + \text{bono}$

DesarrolladorSenior

- Atributo adicional: `horasTrabajadas (int)`
- $\text{Salario} = \text{salarioBase} + (\text{horasTrabajadas} * 35)$

DesarrolladorJunior

- Atributo adicional: `horasExtras (int)`
- $\text{Salario} = \text{salarioBase} + (\text{horasExtras} * 18)$

Interfaz Imprimible

Define el método:

- imprimirDetalles(): void

Este método imprimirá los detalles específicos del empleado y su salario final.

Cada clase de empleado deberá implementar esta interfaz.

Clase Empresa

La clase Empresa gestionará los empleados utilizando un ArrayList.

Atributo:

- ArrayList<Empleado> empleados

Métodos:

- agregarEmpleado(Empleado e): boolean
Añade el empleado a la lista y devuelve true si se ha añadido correctamente.
- imprimirDetallesEmpleados(): void
Recorre la lista e invoca imprimirDetalles() de cada empleado.
- buscarEmpleado(String nombre): Empleado
Devuelve el empleado si existe o null en caso contrario.

Programa Principal

El programa debe incluir un menú interactivo.

Opciones del menú

a) Registrar empleado

- Solicita el tipo de empleado:
 - Socio
 - GeneralManager
 - Manager
 - DesarrolladorSenior
 - DesarrolladorJunior
- Solicita los datos comunes:
 - nombre, edad, salarioBase
- Solicita el dato específico según el tipo.

- Añade el empleado al ArrayList de la empresa.

b) Mostrar detalles de empleados

- Llama a imprimirDetallesEmpleados().

c) Buscar empleado por nombre

- Solicita un nombre.
- Si existe, muestra sus detalles.

d) Salir

- Finaliza el programa.

Requisitos de validación

Se debe validar:

- $\text{edad} \geq 16$
- $\text{salarioBase} > 0$
- $\text{stockOptions}, \text{horasTrabajadas} \text{ y } \text{horasExtras} \geq 0$
- $\text{bono} \geq 0$
- Control de opciones incorrectas del menú.

Rúbrica de Evaluación

Criterio	Descripción	Puntos
Clase Empleado	Implementación correcta de atributos, constructor y métodos comunes.	2
Método Abstracto	Implementación correcta de calcularSalario() en las subclases.	2
Interfaz Imprimible	Creación e implementación correcta en todas las clases.	2
Uso de ArrayList	Gestión correcta de la colección y recorridos.	2
Programa Principal	Menú interactivo funcional.	1
Validación de datos	Control adecuado de entradas del usuario.	1
Total		10